

Universidad Católica de Salta
Licenciatura en Ciencia de Datos

Examen Final Integrador

Programación II

Sistema de control de gestión de sucursal

Alumno: Facundo Luna

Profesor: Ing. Mario Martínez

Año lectivo: 2026

Fecha: 27 de octubre de 2026

Índice

Aclaraciones previas **2**

1. Enunciado del ejercicio **3**

1.1. Planteo del problema 3

1.2. Datos de entrada 3

1.3. Procesamiento solicitado 4

1.4. Interacción con el usuario 4

2. Mejoras respecto del Ejercicio 14 del TP 4 **6**

3. Funcionamiento del sistema **8**

3.1. Pantalla de bienvenida e identificación 8

3.2. Menú principal 8

3.3. Resumen anual (opción 2) 9

3.4. Facturación y merma por sector (opción 3) 9

3.5. Evolución mes a mes (opción 4) 10

3.6. Análisis por trimestre (opción 5) 10

3.7. Consulta de un sector (opción 6) 11

3.8. Informe exportado (opción 9) 11

3.9. Exportación de los datos para análisis (opción 10) 12

4. Gráficos **13**

4.1. Gráficos de facturación (opción 7) 13

4.2. Gráficos de merma (opción 8) 13

5. Código fuente **15**

6. Archivos de datos **42**

6.1. sectores.csv 42

6.2. ventas.csv 42

Aclaraciones previas

El presente trabajo corresponde al examen final integrador de Programación II. La consigna de la cátedra pide tomar como base el Ejercicio 14 del Trabajo Práctico N° 4 y mejorarlo incorporando más herramientas y complejidad, presentando tanto el enunciado del ejercicio como su programación en Python.

Siguiendo la recomendación de la cátedra, se conservó el tema del Ejercicio 14 (el análisis anual de ventas y merma de una sucursal de supermercado) y se lo extendió hasta convertirlo en un sistema de consulta completo. La elección del tema no responde únicamente a esa continuidad: quien suscribe trabaja actualmente en un supermercado, por lo que se trata de un dominio conocido de primera mano. Eso es lo que permitió que el criterio central del trabajo, la meta de merma propia de cada sector, esté tomado de cómo se controla efectivamente la mercadería en una sucursal (no es lo mismo lo que se puede descartar en limpieza que en verdulería) y no de valores elegidos al azar para completar el ejercicio. La sección 2 detalla punto por punto qué se agregó respecto de aquella versión.

El proyecto está formado por los siguientes archivos:

- `Examen_Final.py`: el programa, con el apellido y nombre del alumno en la primera línea de comentario, según el formato exigido por la cátedra.
- `ventas.csv`: el archivo de ventas del Ejercicio 14, con 72 registros (12 meses por 6 sectores).
- `sectores.csv`: archivo nuevo, con la ficha de cada sector.
- `programacion2-examen-final-facundo-luna.pdf`: este documento.

El programa se ejecuta desde la carpeta donde están los dos archivos CSV, ya que los abre por su nombre y no por una ruta absoluta. Requiere las bibliotecas `numpy` y `matplotlib`.

1. Enunciado del ejercicio

1.1. Planteo del problema

Una sucursal de supermercado factura todos los meses, pero una parte de la mercadería nunca llega a venderse: se vence, se rompe o se descarta por mal estado. Esa pérdida se denomina **merma** y, al igual que la facturación, se mide en dinero.

El criterio habitual para controlarla es fijar un límite único de merma para toda la sucursal, que es lo que hacía el Ejercicio 14 del TP 4 con su constante del 3 %. Ese criterio tiene un problema: no todos los sectores enfrentan el mismo riesgo. Un sector de almacén vende mercadería que no se echa a perder, mientras que una verdulería trabaja con productos que se descartan en pocos días. Un límite común deja pasar al almacén que está perdiendo más de lo que debería y castiga a la verdulería que, en realidad, está dentro de lo esperable para su rubro.

El ejercicio consiste entonces en desarrollar un sistema de control de gestión que evalúe a cada sector contra su propia meta.

1.2. Datos de entrada

El sistema debe leer dos archivos CSV.

El primero, `ventas.csv`, tiene una fila por cada mes y sector:

Columna	Tipo	Descripción
`mes`	entero	número de mes, de 1 a 12
`sector`	texto	nombre del sector de la sucursal
`unidades`	entero	unidades vendidas en el mes
`precio_unitario`	real	precio de venta por unidad
`unidades_merma`	entero	unidades perdidas en el mes

El segundo, `sectores.csv`, tiene una fila por sector con su ficha:

Columna	Tipo	Descripción
`sector`	texto	nombre del sector, que vincula con `ventas.csv`
`responsable`	texto	encargado a cargo del sector
`meta_merma`	real	porcentaje de merma admitido para ese sector
`categoria`	texto	perecedero o no perecedero

1.3. Procesamiento solicitado

A partir de esos datos, el sistema debe:

1. Calcular, para cada sector, la facturación anual, la merma anual en dinero y el porcentaje que la merma representa sobre lo facturado:

   $$\text{facturación} = \text{unidades} \times \text{precio unitario} \qquad \text{merma} = \text{unidades de merma} \times \text{precio unitario}$$

   $$\%\ \text{merma} = \frac{\text{merma}}{\text{facturación}} \times 100$$

2. Comparar ese porcentaje contra la meta propia del sector y asignarle un estado:
   - **OK**: la merma real no supera la meta.
   - **ATENCIÓN**: la supera, pero sin llegar a la meta multiplicada por una tolerancia del 20 %.
   - **ALERTA**: supera ese margen de tolerancia.
   - **SIN META**: el sector aparece en el archivo de ventas pero no tiene ficha cargada en el de sectores, de modo que no hay meta contra la cual compararlo. Sus ventas se informan igual.
3. Informar la evolución mes a mes: lo facturado en cada mes, la variación porcentual respecto del mes anterior y el acumulado del año hasta ese mes.
4. Agrupar los doce meses en cuatro trimestres y comparar el último contra el primero, para distinguir un mal mes aislado de una caída sostenida.
5. Permitir consultar un sector en particular y mostrar su ficha completa, incluyendo cuánto dinero se habría evitado perder si hubiera cumplido su meta.
6. Presentar los resultados en gráficos y exportar un informe a un archivo de texto.
7. Exportar los datos ya procesados a un archivo CSV con una fila por mes y sector, para que puedan seguir analizándose fuera del sistema.

1.4. Interacción con el usuario

El sistema debe abrir con una pantalla de bienvenida que identifique al autor y a la materia, pedir el nombre del usuario y el de la sucursal validando que no queden vacíos ni contengan números, y ofrecer un menú que se repita hasta que el usuario decida salir. Toda opción inválida debe informarse con un mensaje claro sin interrumpir la ejecución.

Antes de pedirle cualquier dato al usuario, el sistema debe verificar que los dos archivos CSV estén presentes. Si falta alguno, debe indicar cuál es, dónde debería estar y desde qué carpeta se lo está ejecutando, en lugar de interrumpirse con un error del intérprete.

El mismo criterio vale para los datos incompletos, porque los archivos los cargan personas y no siempre llegan completos. Un sector sin ficha, un sector que no vendió nada en el mes, un archivo de ventas con menos de doce meses o directamente sin registros son situaciones que el sistema debe informar y seguir funcionando, y no motivos para que se corte.

Las dos salidas que genera el sistema, el informe de texto y el CSV de análisis, contienen la merma, las metas y los nombres de los responsables, de modo que son material de gestión interna de la sucursal.

↑ Volver al índice

2. Mejoras respecto del Ejercicio 14 del TP 4

La siguiente tabla compara ambas versiones.

Aspecto	**Ejercicio 14 (TP 4)**	**Examen final**
Archivos de datos	Un solo CSV (`ventas.csv`)	Dos CSV relacionados entre sí por el nombre del sector
Límite de merma	Una constante única del 3 % para todos los sectores	Una meta propia por sector, leída desde el archivo de fichas
Estados	Dos (OK / ALERTA)	Tres (OK / ATENCIÓN / ALERTA), con una tolerancia intermedia
Diccionarios	Tres diccionarios simples	Siete acumuladores más un diccionario de diccionarios con la ficha de cada sector
Interacción	Dos preguntas al inicio y una consulta suelta	Pantalla de bienvenida, pantalla explicativa y menú de diez opciones que se repite hasta salir
Validación de entradas	Control de campo vacío	Campo vacío, nombre con números (`isalpha`), opción no numérica (`isdigit`) y sugerencia de sector por prefijo (`startswith`)
Operaciones con NumPy	`array`, `sum`, `mean`, `argmax`, `argmin`, `where`, `round`	Se agregan `std`, `diff`, `cumsum`, `reshape`, `sum(axis=1)` y `arange`
Análisis temporal	Total, promedio, mejor y peor mes	Se agregan variación mes a mes, acumulado, desvío estándar, conteo de meses en alza y en baja, y apertura por trimestre
Cortes de análisis	Por sector y por mes	Por sector, por mes, por trimestre y por tipo de mercadería
Gráficos	Dos, en una sola figura	Cuatro, en dos figuras separadas, con etiquetas de valor sobre las marcas
Salida a disco	No tiene	Dos exportaciones: informe de texto y CSV de análisis

Aspecto	**Ejercicio 14 (TP 4)**	**Examen final**
Uso del módulo `csv`	Solo lectura, con `DictReader`	Lectura y escritura, agregando `DictWriter` y `writeheader()`
Comportamiento ante un archivo faltante	Se interrumpe con un error del intérprete	Verifica los archivos con `os.path.exists()` antes de empezar e informa cuál falta y dónde debería estar
Tolerancia a datos incompletos	No está contemplada: cualquier dato fuera de lo previsto corta la ejecución	Se contemplan el sector sin ficha cargada (estado `SIN META`), el sector que no vendió en el mes, el año incompleto y el archivo sin registros

El Ejercicio 14 respondía *cuánta merma tuvo cada sector*; este sistema responde *qué sector se desvió de lo que se esperaba de él*, que es la pregunta que efectivamente necesita el encargado para decidir dónde intervenir. Con el límite único del 3 % del TP 4, la carnicería aparecía en alerta y la fiambrería pasaba sin observaciones; evaluando a cada sector contra su propia meta, ambas quedan en atención y la verdulería pasa a ser el único caso realmente grave.

↑ Volver al índice

3. Funcionamiento del sistema

3.1. Pantalla de bienvenida e identificación

Al iniciar, el sistema se presenta y pide los datos del usuario. El nombre se valida con `isalpha()` y la sucursal se normaliza a mayúsculas.

```
========================================================================
                SISTEMA DE CONTROL DE GESTIÓN DE SUCURSAL
                  Análisis anual de facturación y merma
========================================================================
  Autor:     Luna, Facundo
  Materia:   Programación II
  Carrera:   Licenciatura en Ciencia de Datos
  Profesor:  Ing. Mario Martínez
  Año:       2026
  Versión:   1.0
========================================================================

Ingrese su nombre y apellido: Facundo Luna
Ingrese el nombre de la sucursal: Salta Centro

Bienvenido/a al control de gestión, Facundo Luna
Sucursal en análisis: SALTA CENTRO

Se leyeron 72 registros de ventas y 6 fichas de sectores
```

3.2. Menú principal

```
------------------------------------------------------------------------
                             MENÚ PRINCIPAL
------------------------------------------------------------------------
  1. ¿Qué hace este sistema?
  2. Resumen anual de la sucursal
  3. Facturación y merma por sector
  4. Evolución mes a mes
  5. Análisis por trimestre
  6. Consultar un sector en particular
  7. Gráficos de facturación
  8. Gráficos de merma
  9. Generar informe en archivo de texto
 10. Exportar los datos para análisis (CSV)
  0. Salir del sistema
------------------------------------------------------------------------
Elija una opción (0 a 10):
```

Si al iniciar falta alguno de los archivos de datos, el sistema lo informa antes de pedir cualquier dato y se detiene con un mensaje explicativo en lugar de interrumpirse con un error del intérprete:

```
No se puede iniciar el sistema porque faltan archivos de datos:
  - ventas.csv
  - sectores.csv

Los archivos deben estar en la misma carpeta que Examen_Final.py
El programa se está ejecutando en: C:\Users\Facundo\Documents
```

3.3. Resumen anual (opción 2)

```
RESUMEN ANUAL - SUCURSAL SALTA CENTRO
--------------------------------------------------------------------------------
  Meses analizados:                          12
  Sectores analizados:                        6
  Facturación total:          $  3,364,109,400.00
  Merma total:                $     65,819,740.00
  Merma sobre facturación:                  1.96 %
  Promedio mensual:           $    280,342,450.00
  Desvío entre meses:         $     46,422,246.71
  Mejor mes:                              Diciembre
  Peor mes:                               Febrero

  Meses por encima del promedio:
    Julio           $   289.2 millones
    Septiembre      $   294.6 millones
    Octubre         $   313.9 millones
    Noviembre       $   336.6 millones
    Diciembre       $   389.8 millones

  Merma según el tipo de mercadería:
    No perecedero       0.24 %   ($4,066,260.00 perdidos)
    Perecedero          3.75 %   ($61,753,480.00 perdidos)
```

La apertura por tipo de mercadería muestra el resultado más contundente del análisis: los sectores perecederos pierden quince veces más, en proporción, que los no perecederos. Ese contraste es precisamente lo que justifica haber reemplazado el límite único por una meta por sector.

3.4. Facturación y merma por sector (opción 3)

```
FACTURACIÓN Y MERMA POR SECTOR - SUCURSAL SALTA CENTRO
--------------------------------------------------------------------------------
  Sector             Facturación           Merma     Real    Meta    Estado
  Carniceria      890,182,800.00   28,222,270.00    3.17%   2.80    ATENCIÓN
  Almacen         821,457,000.00    2,603,330.00    0.32%   0.40    OK
  Bebidas         492,892,200.00    1,036,510.00    0.21%   0.25    OK
  Fiambreria      438,246,600.00   11,581,900.00    2.64%   2.50    ATENCIÓN
  Limpieza        401,770,800.00      426,420.00    0.11%   0.20    OK
  Verduleria      319,560,000.00   21,949,310.00    6.87%   5.00    ALERTA
--------------------------------------------------------------------------------
  TOTAL         3,364,109,400.00   65,819,740.00    1.96%

  Participación de cada sector en la facturación:
    Carniceria     26.5%  **************************
    Almacen        24.4%  ************************
    Bebidas        14.7%  **************
    Fiambreria     13.0%  *************
    Limpieza       11.9%  ***********
    Verduleria      9.5%  *********
```

3.5. Evolución mes a mes (opción 4)

```
EVOLUCIÓN MES A MES - SUCURSAL SALTA CENTRO
------------------------------------------------------------------------
  Mes               Facturación   Variación          Acumulado
  Enero          238,788,000.00           -     238,788,000.00
  Febrero        216,970,800.00       -9.14%    455,758,800.00
  Marzo          245,993,400.00      +13.38%    701,752,200.00
  Abril          243,363,300.00       -1.07%    945,115,500.00
  Mayo           258,390,000.00       +6.17%  1,203,505,500.00
  Junio          257,326,500.00       -0.41%  1,460,832,000.00
  Julio          289,162,200.00      +12.37%  1,749,994,200.00
  Agosto         279,216,000.00       -3.44%  2,029,210,200.00
  Septiembre     294,558,600.00       +5.49%  2,323,768,800.00
  Octubre        313,938,300.00       +6.58%  2,637,707,100.00
  Noviembre      336,577,500.00       +7.21%  2,974,284,600.00
  Diciembre      389,824,800.00      +15.82%  3,364,109,400.00
------------------------------------------------------------------------
  Meses con suba respecto del anterior:  7
  Meses con baja respecto del anterior:  4
  Mejor variación: +15.82% en Diciembre
  Peor variación:  -9.14% en Febrero
```

3.6. Análisis por trimestre (opción 5)

```
ANÁLISIS POR TRIMESTRE - SUCURSAL SALTA CENTRO
------------------------------------------------------------------------
  Trimestre                      Facturación          Merma    % merma
  1 trimestre (Ene-Mar)        701,752,200.00  16,433,320.00      2.34%
  2 trimestre (Abr-Jun)        759,079,800.00  12,383,440.00      1.63%
  3 trimestre (Jul-Sep)        862,936,800.00  13,326,570.00      1.54%
  4 trimestre (Oct-Dic)      1,040,340,600.00  23,676,410.00      2.28%
------------------------------------------------------------------------
  Mejor trimestre: 4 trimestre (Oct-Dic)
  Peor trimestre:  1 trimestre (Ene-Mar)
  Variacion del 4 trimestre contra el 1: +48.25%
```

El año cierra creciendo un 48,25 % de punta a punta, pero la merma vuelve a subir en el cuarto trimestre: el sistema permite ver que el mayor volumen de venta trajo consigo más pérdida, algo que el promedio anual por sí solo no revelaba.

3.7. Consulta de un sector (opción 6)

```
Ingrese el sector a consultar: Verduleria

FICHA DEL SECTOR VERDULERIA
--------------------------------------------------------------------------
  Responsable:            Luis Vera
  Tipo de mercadería:     Perecedero
  Unidades vendidas:      182,400
  Facturación anual:      $319,560,000.00
  Merma anual:            $21,949,310.00
  Merma real:             6.87 %
  Meta del sector:        5.00 %
  Estado:                 ALERTA
  Participación:          9.5 % de la sucursal

  El sector está 1.87 puntos por encima de su meta
  Si cumpliera la meta se habrían evitado $5,971,310.00 de pérdida
```

Cuando el sector no existe, `startswith()` busca coincidencias por las primeras tres letras y sugiere el nombre correcto en lugar de limitarse a informar el error:

```
Ingrese el sector a consultar: Carn
  El sector Carn no figura en ventas.csv
  Quiso decir Carniceria?
```

3.8. Informe exportado (opción 9)

La opción 9 genera un archivo de texto cuyo nombre se arma a partir del de la sucursal. El informe incluye una sección final con los sectores que requieren revisión y el responsable de cada uno.

```
SECTORES QUE REQUIEREN REVISIÓN
--------------------------------------------------------------------------
Carniceria: 3.17% contra una meta de 2.80% (+0.37 puntos). Responsable: Hugo Paz
Fiambreria: 2.64% contra una meta de 2.50% (+0.14 puntos). Responsable: Ana Sosa
Verduleria: 6.87% contra una meta de 5.00% (+1.87 puntos). Responsable: Luis Vera
```

3.9. Exportación de los datos para análisis (opción 10)

El informe de texto de la opción anterior está escrito para leerse. Esta opción, en cambio, exporta los datos para que puedan seguir procesándose: genera un archivo CSV con una fila por cada mes y sector, es decir, las mismas 72 observaciones del archivo original, pero con los importes ya calculados y con los datos de la ficha de cada sector incorporados.

```
Datos exportados en el archivo analisis_salta_centro.csv
72 filas y 15 columnas
El archivo se puede abrir con Excel o leer con pandas para
seguir analizando los datos fuera de este sistema
```

El archivo se escribe con `csv.DictWriter`, la contraparte del `DictReader` que se usa para leer: en lugar de devolver un diccionario por fila, recibe un diccionario y lo escribe como fila, tomando de la lista de columnas el orden en que va cada dato. Las columnas exportadas son:

Columna	Contenido
`mes`, `nombre_mes`, `trimestre`	el período, en número y en texto
`sector`, `categoria`, `responsable`	la identificación del sector y su ficha
`unidades`, `precio_unitario`	los datos originales de la venta
`facturacion`, `merma`	los importes calculados por el sistema
`unidades_merma`, `porcentaje_merma`	la pérdida de esa fila, en unidades y en porcentaje
`meta_merma`, `desvio`	la meta del sector y los puntos de diferencia
`estado_anual`	el estado que el sistema asignó al sector

El criterio de armado es que cada fila sea una observación y cada columna una variable, que es el formato que esperan las herramientas de análisis. De esa manera el archivo puede abrirse en una planilla de cálculo o leerse con `pandas` y reagruparse por sector, por mes, por trimestre o por categoría sin ninguna preparación previa, en lugar de quedar atado a los cortes que decidió este programa.

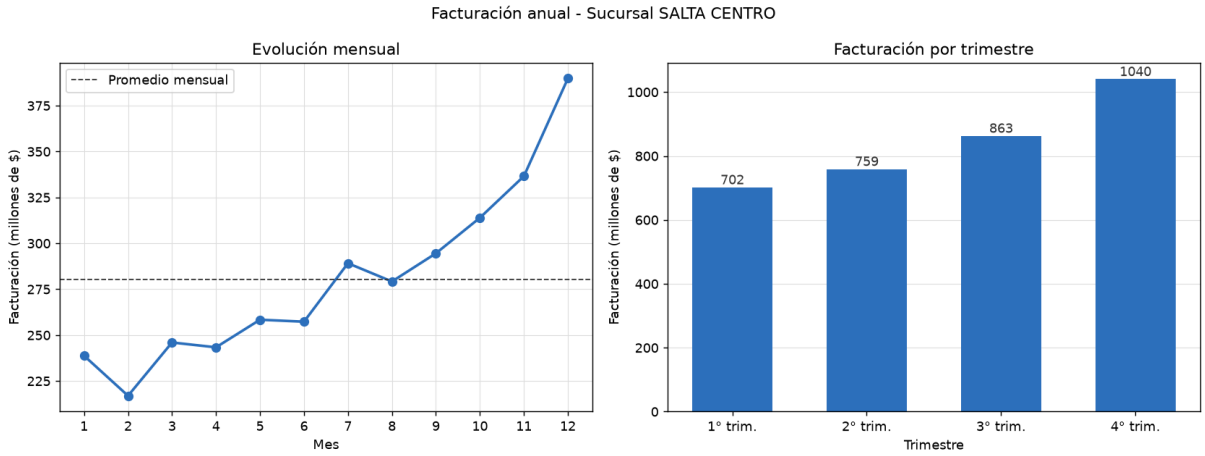
```
mes,nombre_mes,trimestre,sector,categoria,responsable,unidades,...
1,Enero,1,Almacen,No perecedero,Marta Gómez,48600,1200.0,58320000.0,...
1,Enero,1,Bebidas,No perecedero,Diego Ruiz,19440,1800.0,34992000.0,...
1,Enero,1,Carniceria,Perecedero,Hugo Paz,9720,6500.0,63180000.0,...
```

↑ Volver al índice

4. Gráficos

4.1. Gráficos de facturación (opción 7)

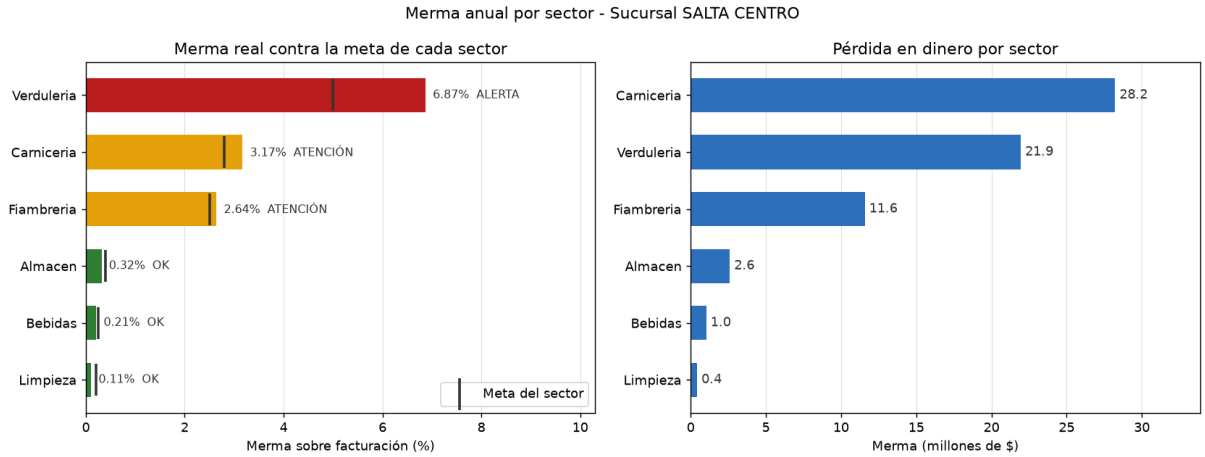
La primera figura reúne la evolución mensual y el resumen por trimestre. En el gráfico de línea, la línea de trazos marca el promedio mensual, de modo que se distingue a simple vista qué meses lo superan. En el de barras, el valor va escrito sobre cada barra para no tener que seguir la línea del eje.



4.2. Gráficos de merma (opción 8)

La segunda figura compara la merma de cada sector contra su meta y, por separado, la pérdida expresada en dinero. Las barras son horizontales para que los nombres de los sectores se lean sin rotar, y cada meta se marca con un trazo vertical sobre su propia barra: la barra que pasa su marca es la que se desvió.

El color indica el estado, pero nunca por sí solo: junto a cada barra se escriben el porcentaje y el estado en palabras, de manera que el gráfico siga siendo legible impreso en blanco y negro o para quien no distinga el rojo del verde.



Los dos paneles están en unidades distintas (porcentaje uno, pesos el otro) y por eso se presentan como gráficos separados y no superpuestos en un mismo par de ejes. La comparación entre ambos es justamente el punto: la carnicería es la que más dinero pierde en términos absolutos, pero la verdulería, que pierde menos plata, es la que está peor respecto de lo que se espera de ella.

↑ Volver al índice

5. Código fuente

A continuación se transcribe el contenido completo de `Examen_Final.py`.

```
# Luna, Facundo

# EXAMEN FINAL INTEGRADOR - PROGRAMACION II
# Sistema de control de gestion anual de una sucursal de supermercado
#
# Lee ventas.csv (una fila por mes y sector, con unidades vendidas, precio
# unitario y unidades perdidas) y sectores.csv (la ficha de cada sector, con su
# responsable, su meta de merma y su categoria de mercaderia)
#
# Calcula la facturacion y la merma de cada sector, compara la merma contra la
# meta particular de cada uno en lugar de un unico limite general, mide la
# evolucion mes a mes y por trimestre, y presenta todo en un menu de consulta
# con tablas, graficos, un informe de texto y una exportacion para analisis

import csv
import os
import sys
import numpy as np
import matplotlib.pyplot as plt


# ============================================================
# CONSTANTES DEL SISTEMA
# ============================================================

AUTOR = "Luna, Facundo"
MATERIA = "Programación II"
CARRERA = "Licenciatura en Ciencia de Datos"
PROFESOR = "Ing. Mario Martínez"
ANIO_LECTIVO = "2026"
VERSION = "1.0"

# Ancho en caracteres de las pantallas, para que los recuadros y los titulos
```

```
# centrados queden alineados entre si
ANCHO = 74

# Si la merma supera la meta del sector pero no llega a la meta multiplicada
# por este factor, se considera una desviacion leve (ATENCION) en vez de ALERTA
TOLERANCIA = 1.2

ARCHIVO_VENTAS = "ventas.csv"
ARCHIVO_SECTORES = "sectores.csv"

NOMBRES_MESES = {
    1: "Enero", 2: "Febrero", 3: "Marzo",
    4: "Abril", 5: "Mayo", 6: "Junio",
    7: "Julio", 8: "Agosto", 9: "Septiembre",
    10: "Octubre", 11: "Noviembre", 12: "Diciembre",
}

NOMBRES_TRIMESTRES = {
    1: "1° trimestre (Ene-Mar)",
    2: "2° trimestre (Abr-Jun)",
    3: "3° trimestre (Jul-Sep)",
    4: "4° trimestre (Oct-Dic)",
}

# Tonos que se distinguen tambien en blanco y negro. De todas formas el estado
# nunca se informa solo con el color, siempre va acompanado del texto
COLOR_ESTADO = {
    "OK": "#2e7d32",
    "ATENCIÓN": "#e3a008",
    "ALERTA": "#b91c1c",
    "SIN META": "#6b7280",
}

COLOR_FACTURACION = "#2c6fbb"
COLOR_REFERENCIA = "#333333"


# ============================================================
# PANTALLA DE BIENVENIDA
# ============================================================

```

```
75  print()
76  print("=" * ANCHO)
77  print("SISTEMA DE CONTROL DE GESTIÓN DE SUCURSAL".center(ANCHO))
78  print("Análisis anual de facturación y merma".center(ANCHO))
79  print("=" * ANCHO)
80  print(f"  Autor:      {AUTOR}")
81  print(f"  Materia:    {MATERIA}")
82  print(f"  Carrera:    {CARRERA}")
83  print(f"  Profesor:   {PROFESOR}")
84  print(f"  Año:        {ANIO_LECTIVO}")
85  print(f"  Versión:    {VERSION}")
86  print("=" * ANCHO)
87  print()
88
89
90  # ============================================================
91  # CONTROL DE LOS ARCHIVOS DE DATOS
92  # ============================================================
93
94  # Se controla antes de pedirle nada al usuario, para que no
        complete sus datos
95  # y recien despues se entere de que falta un archivo
96  archivos_faltantes = []
97
98  for archivo_necesario in [ARCHIVO_VENTAS, ARCHIVO_SECTORES]:
99      if not os.path.exists(archivo_necesario):
100         archivos_faltantes.append(archivo_necesario)
101
102 if len(archivos_faltantes) > 0:
103     print("No se puede iniciar el sistema porque faltan archivos
           de datos:")
104
105     for archivo_faltante in archivos_faltantes:
106         print(f"  - {archivo_faltante}")
107
108     print()
109     print("Los archivos deben estar en la misma carpeta que
           Examen_Final.py")
110     print(f"El programa se está ejecutando en: {os.getcwd()}")
111     print()
112
113     sys.exit()
114
115
116 # ============================================================
```

```python
# IDENTIFICACION DEL USUARIO
# ============================================================

# replace() saca los espacios interiores e isalpha() comprueba que lo que quedo
# sean solo letras, asi no se acepta un nombre con numeros
nombre = ""

while nombre == "":
    nombre = input("Ingrese su nombre y apellido: ").strip().title()

    if nombre == "":
        print("  El nombre no puede quedar vacío")
    elif not nombre.replace(" ", "").isalpha():
        print("  El nombre solo puede contener letras")
        nombre = ""

sucursal = ""

while sucursal == "":
    sucursal = input("Ingrese el nombre de la sucursal: ").strip().upper()

    if sucursal == "":
        print("  La sucursal no puede quedar vacía")

# Iniciales con las que se firma el informe
letras_iniciales = []

for palabra in nombre.split():
    letras_iniciales.append(palabra[0])

iniciales = "".join(letras_iniciales)

print()
print(f"Bienvenido/a al control de gestión, {nombre}")
print(f"Sucursal en análisis: {sucursal}")


# ============================================================
# LECTURA DE LOS ARCHIVOS CSV
# ============================================================

# Se indica utf-8 porque sectores.csv tiene nombres con tilde y
```

```
    sin ese dato
# Windows los leeria con otra codificacion
#
# A cada sector le corresponde un diccionario con su ficha, de
    modo que
# fichas_sectores es un diccionario de diccionarios
fichas_sectores = {}

archivo = open(ARCHIVO_SECTORES, "r", encoding="utf-8")
datos_sectores = list(csv.DictReader(archivo))
archivo.close()

for fila in datos_sectores:
    sector = fila["sector"].strip().capitalize()

    fichas_sectores[sector] = {
        "responsable": fila["responsable"].strip().title(),
        "meta_merma": float(fila["meta_merma"]),
        "categoria": fila["categoria"].strip().capitalize(),
    }

archivo = open(ARCHIVO_VENTAS, "r", encoding="utf-8")
datos_ventas = list(csv.DictReader(archivo))
archivo.close()

# El archivo puede existir y tener solo el encabezado. Sin
    ninguna fila no hay
# nada que calcular, asi que conviene avisarlo aca y no dejar
    que el programa
# siga hasta fallar con los acumuladores vacios
if len(datos_ventas) == 0:
    print()
    print(f"El archivo {ARCHIVO_VENTAS} no tiene registros de
        ventas cargados")
    print("Debe tener al menos una fila además del encabezado")
    print()

    sys.exit()

print()
print(f"Se leyeron {len(datos_ventas)} registros de ventas "
      f"y {len(fichas_sectores)} fichas de sectores")


# ============================================================
```

```
# CALCULOS SOBRE LOS DATOS LEIDOS
# ============================================================

ventas_por_sector = {}
merma_por_sector = {}
unidades_por_sector = {}
ventas_por_mes = {}
merma_por_mes = {}
ventas_por_categoria = {}
merma_por_categoria = {}

# Copia de cada fila con sus importes ya calculados, que es lo que exporta la
# opcion 10
detalle_filas = []

filas_sin_ficha = 0

# El if/else resuelve el arranque de cada acumulador: si la clave todavia no
# figura en el diccionario no se le puede sumar nada, asi que la primera vez se
# asigna el valor y recien a partir de la segunda se acumula
for fila in datos_ventas:
    mes = int(fila["mes"])
    sector = fila["sector"].strip().capitalize()
    unidades = int(fila["unidades"])
    precio_unitario = float(fila["precio_unitario"])
    unidades_merma = int(fila["unidades_merma"])

    facturacion = unidades * precio_unitario
    merma = unidades_merma * precio_unitario

    # Un sector puede no haber vendido nada en el mes, por ejemplo si estuvo
    # cerrado por refacciones. En ese caso no hay facturacion sobre la cual
    # calcular el porcentaje, y se deja en cero para no dividir por cero
    if facturacion > 0:
        porcentaje_merma_fila = round(merma / facturacion * 100, 2)
    else:
        porcentaje_merma_fila = 0.0

```

```
237     # El // es la division entera: los meses 1 a 3 caen en el
            trimestre 1, los
238     # meses 4 a 6 en el 2, y asi con el resto
239     detalle_filas.append({
240         "mes": mes,
241         "nombre_mes": NOMBRES_MESES[mes],
242         "trimestre": (mes - 1) // 3 + 1,
243         "sector": sector,
244         "unidades": unidades,
245         "precio_unitario": precio_unitario,
246         "facturacion": round(facturacion, 2),
247         "unidades_merma": unidades_merma,
248         "merma": round(merma, 2),
249         "porcentaje_merma": porcentaje_merma_fila,
250     })
251
252     if sector in ventas_por_sector:
253         ventas_por_sector[sector] = ventas_por_sector[sector] +
                facturacion
254         merma_por_sector[sector] = merma_por_sector[sector] +
                merma
255         unidades_por_sector[sector] =
                unidades_por_sector[sector] + unidades
256     else:
257         ventas_por_sector[sector] = facturacion
258         merma_por_sector[sector] = merma
259         unidades_por_sector[sector] = unidades
260
261     if mes in ventas_por_mes:
262         ventas_por_mes[mes] = ventas_por_mes[mes] + facturacion
263         merma_por_mes[mes] = merma_por_mes[mes] + merma
264     else:
265         ventas_por_mes[mes] = facturacion
266         merma_por_mes[mes] = merma
267
268     # La categoria no esta en ventas.csv sino en la ficha, y el
            in evita el
269     # error cuando un sector no la tiene cargada
270     if sector in fichas_sectores:
271         categoria = fichas_sectores[sector]["categoria"]
272
273         if categoria in ventas_por_categoria:
274             ventas_por_categoria[categoria] =
                    ventas_por_categoria[categoria] + facturacion
275             merma_por_categoria[categoria] =
```

```
                merma_por_categoria[categoria] + merma
276         else:
277             ventas_por_categoria[categoria] = facturacion
278             merma_por_categoria[categoria] = merma
279     else:
280         filas_sin_ficha = filas_sin_ficha + 1
281
282 if filas_sin_ficha > 0:
283     print(f"Aviso: {filas_sin_ficha} registros corresponden a
        sectores "
284           f"sin ficha en {ARCHIVO_SECTORES}")
285
286 # Porcentaje de merma de cada sector y estado que resulta de
    compararlo contra
287 # la meta propia del sector
288 porcentajes_merma = {}
289 estados_sector = {}
290
291 for sector in ventas_por_sector:
292     # Mismo cuidado que en el detalle: un sector sin ventas en
        todo el ano no
293     # tiene facturacion contra la cual medir la merma
294     if ventas_por_sector[sector] > 0:
295         porcentaje = merma_por_sector[sector] /
              ventas_por_sector[sector] * 100
296     else:
297         porcentaje = 0.0
298
299     porcentajes_merma[sector] = porcentaje
300
301     if sector in fichas_sectores:
302         meta = fichas_sectores[sector]["meta_merma"]
303
304         if porcentaje <= meta:
305             estados_sector[sector] = "OK"
306         elif porcentaje <= meta * TOLERANCIA:
307             estados_sector[sector] = "ATENCIÓN"
308         else:
309             estados_sector[sector] = "ALERTA"
310     else:
311         estados_sector[sector] = "SIN META"
312
313 # A cada fila del detalle se le suman los datos que no venian en
    ventas.csv,
314 # para que el archivo exportado quede completo por si solo
```

```
315 for registro in detalle_filas:
316     sector_registro = registro["sector"]
317 
318     if sector_registro in fichas_sectores:
319         meta_registro =
               fichas_sectores[sector_registro]["meta_merma"]
320 
321         registro["categoria"] =
               fichas_sectores[sector_registro]["categoria"]
322         registro["responsable"] =
               fichas_sectores[sector_registro]["responsable"]
323         registro["meta_merma"] = meta_registro
324         registro["desvio"] = round(registro["porcentaje_merma"]
               - meta_registro, 2)
325     else:
326         registro["categoria"] = ""
327         registro["responsable"] = ""
328         registro["meta_merma"] = ""
329         registro["desvio"] = ""
330 
331     registro["estado_anual"] = estados_sector[sector_registro]
332 
333 # El signo menos invierte el orden, porque sorted() ordena de
       menor a mayor
334 sectores_por_facturacion = sorted(ventas_por_sector,
335                                   key=lambda s:
                                      -ventas_por_sector[s])
336 
337 # Los meses se ordenan porque un diccionario no garantiza ningun
       orden. Con los
338 # importes ordenados se arman los arrays, que permiten calcular
       sobre toda la
339 # serie de una sola vez, sin recorrerla con un bucle
340 meses = sorted(ventas_por_mes)
341 
342 importes = []
343 mermas = []
344 
345 for mes in meses:
346     importes.append(ventas_por_mes[mes])
347     mermas.append(merma_por_mes[mes])
348 
349 importes = np.array(importes)
350 mermas = np.array(mermas)
351 
```

```
352 # std() mide cuanto se apartan los meses del promedio: cuanto
        mas alto, mas
353 # desparejo fue el ano. argmax() y argmin() no devuelven el
        importe sino la
354 # POSICION donde esta, y con esa posicion se obtiene el mes
355 total_anual = importes.sum()
356 merma_anual = mermas.sum()
357 promedio_mensual = importes.mean()
358 desvio_mensual = importes.std()
359 mejor_mes = meses[int(importes.argmax())]
360 peor_mes = meses[int(importes.argmin())]
361 porcentaje_merma_anual = merma_anual / total_anual * 100
362
363 # where() conserva el importe en los meses que superan el
        promedio y pone cero
364 # en el resto
365 sobre_promedio = np.where(importes > promedio_mensual, importes,
        0)
366
367 # diff() devuelve la diferencia entre cada mes y el anterior,
        por eso tiene un
368 # elemento menos que la serie
369 diferencias = np.diff(importes)
370 variaciones = diferencias / importes[:-1] * 100
371
372 acumulado = importes.cumsum()
373
374 # reshape() reacomoda los doce meses en cuatro filas de tres y
        sum(axis=1) suma
375 # cada fila, es decir cada trimestre
376 hay_ano_completo = len(meses) == 12
377
378 if hay_ano_completo:
379     trimestres = importes.reshape(4, 3).sum(axis=1)
380     trimestres_merma = mermas.reshape(4, 3).sum(axis=1)
381 else:
382     trimestres = np.array([])
383     trimestres_merma = np.array([])
384
385
386 # ============================================================
387 # MENU PRINCIPAL
388 # ============================================================
389
390 # isdigit() comprueba que lo ingresado sean solo digitos, asi
```

```
    que escribir una
# letra no interrumpe el programa sino que vuelve a preguntar
opcion = ""

while opcion != "0":
    print()
    print("-" * ANCHO)
    print("MENÚ PRINCIPAL".center(ANCHO))
    print("-" * ANCHO)
    print("  1. ¿Qué hace este sistema?")
    print("  2. Resumen anual de la sucursal")
    print("  3. Facturación y merma por sector")
    print("  4. Evolución mes a mes")
    print("  5. Análisis por trimestre")
    print("  6. Consultar un sector en particular")
    print("  7. Gráficos de facturación")
    print("  8. Gráficos de merma")
    print("  9. Generar informe en archivo de texto")
    print(" 10. Exportar los datos para análisis (CSV)")
    print("  0. Salir del sistema")
    print("-" * ANCHO)

    opcion = input("Elija una opción (0 a 10): ").strip()

    if not opcion.isdigit():
        print("\n  Debe ingresar un número del 0 al 10")
        opcion = ""

    #
       ------------------------------------------------------------
    # 1. PANTALLA EXPLICATIVA DEL PROYECTO
    #
       ------------------------------------------------------------
    elif opcion == "1":
        print()
        print("=" * ANCHO)
        print("¿QUÉ HACE ESTE SISTEMA?".center(ANCHO))
        print("=" * ANCHO)
        print("""
  Una sucursal de supermercado factura todos los meses, pero una
     parte de
  la mercadería nunca llega a venderse: se vence, se rompe o se
     descarta
  por mal estado. Esa pérdida se llama MERMA y se mide en
     dinero, igual
```

```
  que la facturación.

  El problema es que la merma no se puede juzgar con un único límite para
  toda la sucursal. Un sector de almacén, donde la mercadería no se echa
  a perder, no puede compararse con una verdulería. Por eso cada sector
  tiene cargada su propia meta en el archivo de fichas.

  El sistema entonces:

    - Lee las ventas del año y la ficha de cada sector desde dos archivos
      CSV separados.
    - Calcula cuánto facturó y cuánto perdió cada sector, y qué porcentaje
      representa esa pérdida sobre lo que facturó.
    - Compara ese porcentaje contra la meta propia del sector y le asigna
      un estado: OK, ATENCIÓN o ALERTA.
    - Mide cómo evolucionó la facturación mes a mes y por trimestre, para
      distinguir un mal mes puntual de una caída sostenida.
    - Muestra los resultados en tablas y en gráficos, y los exporta a un
      informe de texto y a un archivo de datos para seguir analizándolos.

  El objetivo final es que el encargado sepa, sin revisar planilla por
  planilla, qué sector necesita atención y en qué momento del año se le
  fue de las manos.
""")
        print("=" * ANCHO)

    #
       ------------------------------------------------------------
    # 2. RESUMEN ANUAL
    #
       ------------------------------------------------------------
    elif opcion == "2":
        print()
        print(f"RESUMEN ANUAL - SUCURSAL {sucursal}")
```

```
462         print("-" * ANCHO)
463         print(f"  Meses analizados:          {len(meses):>19}")
464         print(f"  Sectores analizados:
                {len(ventas_por_sector):>19}")
465         print(f"  Facturación total:
                ${total_anual:>18,.2f}")
466         print(f"  Merma total:
                ${merma_anual:>18,.2f}")
467         print(f"  Merma sobre facturación:
                {porcentaje_merma_anual:>18.2f} %")
468         print(f"  Promedio mensual:
                ${promedio_mensual:>18,.2f}")
469         print(f"  Desvío entre meses:
                ${desvio_mensual:>18,.2f}")
470         print(f"  Mejor mes:
                {NOMBRES_MESES[mejor_mes]:>19}")
471         print(f"  Peor mes:
                {NOMBRES_MESES[peor_mes]:>19}")
472
473         # Los ceros que dejo where() son los meses que no
                superan el promedio
474         print()
475         print("  Meses por encima del promedio:")
476
477         for i in range(len(meses)):
478             if sobre_promedio[i] > 0:
479                 nombre_mes = NOMBRES_MESES[meses[i]]
480                 millones = sobre_promedio[i] / 1000000
481                 print(f"    {nombre_mes.ljust(12)}
                        ${millones:>8.1f} millones")
482
483         print()
484         print("  Merma según el tipo de mercadería:")
485
486         for categoria in ventas_por_categoria:
487             facturado = ventas_por_categoria[categoria]
488             perdido = merma_por_categoria[categoria]
489             print(f"    {categoria.ljust(16)} {perdido /
                    facturado * 100:>6.2f} %"
490                   f"   (${perdido:,.2f} perdidos)")
491
492     #
            ------------------------------------------------------------
493     # 3. TABLA POR SECTOR
494     #
```

```
        ------------------------------------------------------------
    elif opcion == "3":
        # ljust() alinea los nombres a la izquierda y los :> alinean los
        # numeros a la derecha, para que las columnas queden parejas
        print()
        print(f"FACTURACIÓN Y MERMA POR SECTOR - SUCURSAL {sucursal}")
        print("-" * ANCHO)
        print(f"  {'Sector'.ljust(12)} {'Facturación':>16} {'Merma':>14} "
              f"{'Real':>7} {'Meta':>6}   Estado")

        for sector in sectores_por_facturacion:
            facturado = ventas_por_sector[sector]
            perdido = merma_por_sector[sector]
            porcentaje = porcentajes_merma[sector]
            estado = estados_sector[sector]

            if sector in fichas_sectores:
                meta = f"{fichas_sectores[sector]['meta_merma']:.2f}"
            else:
                meta = "-"

            print(f"  {sector.ljust(12)} {facturado:>16,.2f} {perdido:>14,.2f} "
                  f"{porcentaje:>6.2f}% {meta:>6}   {estado}")

        print("-" * ANCHO)
        print(f"  {'TOTAL'.ljust(12)} {total_anual:>16,.2f} {merma_anual:>14,.2f} "
              f"{porcentaje_merma_anual:>6.2f}%")

        print()
        print("  Participación de cada sector en la facturación:")

        for sector in sectores_por_facturacion:
            participacion = ventas_por_sector[sector] / total_anual * 100
            barra = "*" * int(participacion)
            print(f"    {sector.ljust(12)} {participacion:>5.1f}%  {barra}")
```

```
529
530     #
        ------------------------------------------------------------
531     # 4. EVOLUCION MES A MES
532     #
        ------------------------------------------------------------
533     elif opcion == "4":
534         print()
535         print(f"EVOLUCIÓN MES A MES - SUCURSAL {sucursal}")
536         print("-" * ANCHO)
537         print(f"  {'Mes'.ljust(12)} {'Facturación':>16}
                {'Variación':>11} "
538               f"{'Acumulado':>18}")
539
540         for i in range(len(meses)):
541             nombre_mes = NOMBRES_MESES[meses[i]]
542
543             # El primer mes no tiene con que compararse. Del
                    segundo en
544             # adelante se usa variaciones[i - 1], corrida una
                    posicion porque
545             # diff() devuelve un elemento menos
546             if i == 0:
547                 variacion = "-"
548             else:
549                 variacion = f"{variaciones[i - 1]:+.2f}%"
550
551             print(f"  {nombre_mes.ljust(12)}
                    {importes[i]:>16,.2f} "
552                   f"{variacion:>11} {acumulado[i]:>18,.2f}")
553
554         meses_en_alza = int((diferencias > 0).sum())
555         meses_en_baja = int((diferencias < 0).sum())
556
557         print("-" * ANCHO)
558         print(f"  Meses con suba respecto del anterior:
                {meses_en_alza}")
559         print(f"  Meses con baja respecto del anterior:
                {meses_en_baja}")
560
561         # Con un solo mes cargado no hay variaciones que
                comparar, porque
562         # diff() necesita dos valores para calcular una
                diferencia
563         if len(variaciones) == 0:
```

```
564             print("  Se necesitan al menos dos meses para comparar variaciones")
565         else:
566             print(f"  Mejor variación: {variaciones.max():+.2f}% en "
567                   f"{NOMBRES_MESES[meses[int(variaciones.argmax()) + 1]]}")
568             print(f"  Peor variación:  {variaciones.min():+.2f}% en "
569                   f"{NOMBRES_MESES[meses[int(variaciones.argmin()) + 1]]}")
570
571     # ------------------------------------------------------------------
572     # 5. ANALISIS POR TRIMESTRE
573     # ------------------------------------------------------------------
574     elif opcion == "5":
575         print()
576         print(f"ANÁLISIS POR TRIMESTRE - SUCURSAL {sucursal}")
577         print("-" * ANCHO)
578
579         if not hay_ano_completo:
580             print(f"  Se necesitan los 12 meses y el archivo tiene {len(meses)}")
581         else:
582             print(f"  {'Trimestre'.ljust(24)} {'Facturación':>18} "
583                   f"{'Merma':>14} {'% merma':>9}")
584
585             for i in range(len(trimestres)):
586                 nombre_trimestre = NOMBRES_TRIMESTRES[i + 1]
587                 porcentaje = trimestres_merma[i] / trimestres[i] * 100
588                 print(f"  {nombre_trimestre.ljust(24)} {trimestres[i]:>18,.2f} "
589                       f"{trimestres_merma[i]:>14,.2f} {porcentaje:>8.2f}%")
590
591             print("-" * ANCHO)
592             print(f"  Mejor trimestre: "
593                   f"{NOMBRES_TRIMESTRES[int(trimestres.argmax()) + 1]}")
594             print(f"  Peor trimestre:  "
595                   f"{NOMBRES_TRIMESTRES[int(trimestres.argmin())
```

```
                    + 1]}")
596
597             # Comparar el ultimo trimestre contra el primero
                    resume el ano en
598             # un solo numero
599             variacion_anual = (trimestres[3] - trimestres[0]) /
                    trimestres[0] * 100
600             print(f"  Variación del 4° trimestre contra el 1°: "
601                   f"{variacion_anual:+.2f}%")
602
603     #
            ------------------------------------------------------------
604     # 6. CONSULTA DE UN SECTOR
605     #
            ------------------------------------------------------------
606     elif opcion == "6":
607         print()
608         print("SECTORES DISPONIBLES")
609         print("-" * ANCHO)
610
611         for sector in sorted(ventas_por_sector):
612             print(f"  - {sector}")
613
614         consulta = input("\nIngrese el sector a consultar:
                ").strip().capitalize()
615
616         if consulta == "":
617             print("  No se consultó ningún sector")
618         elif consulta in ventas_por_sector:
619             facturado = ventas_por_sector[consulta]
620             perdido = merma_por_sector[consulta]
621             unidades_vendidas = unidades_por_sector[consulta]
622
623             # El sector figura en ventas.csv, pero su ficha
                    puede no estar
624             # cargada en sectores.csv. En ese caso no hay
                    responsable ni meta
625             # que mostrar, y esos renglones se completan con un
                    guion en lugar
626             # de interrumpir la consulta
627             if consulta in fichas_sectores:
628                 responsable =
                        fichas_sectores[consulta]["responsable"]
629                 categoria =
                        fichas_sectores[consulta]["categoria"]
```

```
630                 meta =
                        f"{fichas_sectores[consulta]['meta_merma']:.2f}
                        %"
631             else:
632                 responsable = "-"
633                 categoria = "-"
634                 meta = "-"
635
636             print()
637             print(f"FICHA DEL SECTOR {consulta.upper()}")
638             print("-" * ANCHO)
639             print(f"  Responsable:            {responsable}")
640             print(f"  Tipo de mercadería:     {categoria}")
641             print(f"  Unidades vendidas:
                    {unidades_vendidas:,}")
642             print(f"  Facturación anual:      ${facturado:,.2f}")
643             print(f"  Merma anual:            ${perdido:,.2f}")
644             print(f"  Merma real:
                    {porcentajes_merma[consulta]:.2f} %")
645             print(f"  Meta del sector:        {meta}")
646             print(f"  Estado:
                    {estados_sector[consulta]}")
647             print(f"  Participación:          "
648                   f"{facturado / total_anual * 100:.1f} % de la
                          sucursal")
649
650             # El desvio dice en cuantos puntos porcentuales el
                    sector se pasa
651             # de su meta, que es el dato con el que se decide si
                    intervenir
652             if consulta not in fichas_sectores:
653                 print(f"\n  El sector no tiene ficha cargada en "
654                       f"{ARCHIVO_SECTORES}, así que no hay meta
                             contra la cual "
655                       f"compararlo")
656             else:
657                 desvio = (porcentajes_merma[consulta]
658                           -
                              fichas_sectores[consulta]["meta_merma"])
659
660                 if desvio <= 0:
661                     print(f"\n  El sector está {abs(desvio):.2f}
                            puntos por debajo "
662                           f"de su meta: no requiere
                                 intervención")
```

```
663                 else:
664                     print(f"\n  El sector está {desvio:.2f}
                            puntos por encima de su "
665                           f"meta")
666                     print(f"  Si cumpliera la meta se habrían
                            evitado "
667                           f"${desvio / 100 * facturado:,.2f} de
                              pérdida")
668         else:
669             # startswith() rescata la consulta cuando se
                escribio solo el
670             # comienzo del nombre o se erro el final
671             parecidos = []
672
673             for sector in ventas_por_sector:
674                 if sector.startswith(consulta[:3]):
675                     parecidos.append(sector)
676
677             print(f"  El sector {consulta} no figura en
                {ARCHIVO_VENTAS}")
678
679             if len(parecidos) > 0:
680                 print(f"  ¿Quiso decir {', '.join(parecidos)}?")
681
682     #
            ------------------------------------------------------------
683     # 7. GRAFICOS DE FACTURACION
684     #
            ------------------------------------------------------------
685     elif opcion == "7":
686         print("\n  Abriendo los gráficos de facturación...")
687         print("  Cierre la ventana del gráfico para volver al
                menú")
688
689         # subplots(1, 2) crea una figura con dos graficos lado a
                lado y
690         # devuelve uno por variable, para configurarlos por
                separado
691         figura, (grafico_meses, grafico_trimestres) =
                plt.subplots(1, 2, figsize=(13, 5))
692         figura.suptitle(f"Facturación anual - Sucursal
                {sucursal}")
693
694         # La division por un millon evita que el eje quede con
                numeros de
```

```
695         # nueve cifras
696         importes_en_millones = importes / 1000000
697 
698         grafico_meses.plot(meses, importes_en_millones,
                color=COLOR_FACTURACION,
699                            linewidth=2, zorder=2)
700         grafico_meses.scatter(meses, importes_en_millones,
701                               color=COLOR_FACTURACION, s=40,
                                  zorder=3)
702 
703         # La linea del promedio deja ver de un vistazo que meses
                lo superan
704         grafico_meses.axhline(promedio_mensual / 1000000,
705                               color=COLOR_REFERENCIA,
                                  linestyle="--",
706                               linewidth=1, label="Promedio
                                  mensual")
707 
708         grafico_meses.set_title("Evolución mensual")
709         grafico_meses.set_xlabel("Mes")
710         grafico_meses.set_ylabel("Facturación (millones de $)")
711 
712         # Sin set_xticks matplotlib pondria valores intermedios
                como 2.5
713         grafico_meses.set_xticks(meses)
714         grafico_meses.grid(True, color="#dddddd", linewidth=0.6)
715         grafico_meses.set_axisbelow(True)
716         grafico_meses.legend()
717 
718         if hay_ano_completo:
719             etiquetas_trimestres = ["1° trim.", "2° trim.", "3°
                    trim.", "4° trim."]
720             trimestres_en_millones = trimestres / 1000000
721 
722             grafico_trimestres.bar(etiquetas_trimestres,
                    trimestres_en_millones,
723                                    color=COLOR_FACTURACION,
                                       width=0.6, zorder=2)
724 
725             # El valor escrito sobre cada barra evita tener que
                    seguir la
726             # linea del eje para leerlo
727             for i in range(len(trimestres_en_millones)):
728                 grafico_trimestres.text(i,
                        trimestres_en_millones[i],
```

```
                                        f"{trimestres_en_millones[i]:.0f}",
                                        ha="center", va="bottom",
                                        color=COLOR_REFERENCIA)

            grafico_trimestres.set_title("Facturación por
               trimestre")
            grafico_trimestres.set_xlabel("Trimestre")
            grafico_trimestres.set_ylabel("Facturación (millones
               de $)")
            grafico_trimestres.grid(True, axis="y",
               color="#dddddd", linewidth=0.6)
            grafico_trimestres.set_axisbelow(True)
        else:
            grafico_trimestres.set_title("Facturación por
               trimestre")
            grafico_trimestres.text(0.5, 0.5, "Se necesita el
               año completo",
                                    ha="center", va="center")

        plt.tight_layout()
        plt.show()

    #
       ------------------------------------------------------------
    # 8. GRAFICOS DE MERMA
    #
       ------------------------------------------------------------
    elif opcion == "8":
        print("\n  Abriendo los gráficos de merma...")
        print("  Cierre la ventana del gráfico para volver al
           menú")

        figura, (grafico_metas, grafico_pesos) = plt.subplots(1,
           2, figsize=(13, 5))
        figura.suptitle(f"Merma anual por sector - Sucursal
           {sucursal}")

        # Ordenados por merma, el que mas se desvia queda arriba
           de todo
        sectores_por_merma = sorted(porcentajes_merma,
                                    key=lambda s:
                                       porcentajes_merma[s])

        porcentajes = []
        metas = []
```

```
762         colores = []
763         textos = []
764 
765         for sector in sectores_por_merma:
766             porcentajes.append(porcentajes_merma[sector])
767             colores.append(COLOR_ESTADO[estados_sector[sector]])
768             textos.append(f"{porcentajes_merma[sector]:.2f}%  "
769                           f"{estados_sector[sector]}")
770 
771             # Al sector sin ficha se le carga un nan (no es un
                    numero) en lugar
772             # de un cero, porque un cero se dibujaria como una
                    meta de 0% que
773             # no existe. Matplotlib directamente no dibuja los
                    nan
774             if sector in fichas_sectores:
775                 metas.append(fichas_sectores[sector]["meta_merma"])
776             else:
777                 metas.append(np.nan)
778 
779         # barh() dibuja las barras en horizontal, que permite
                que los nombres
780         # de los sectores se lean derechos y sin rotar
781         posiciones = np.arange(len(sectores_por_merma))
782 
783         grafico_metas.barh(posiciones, porcentajes,
                color=colores, height=0.6,
784                            zorder=2)
785 
786         # Cada meta se marca con un trazo sobre su propia barra:
                la barra que
787         # pasa su marca es la que se desvio
788         grafico_metas.scatter(metas, posiciones, marker="|",
                s=600,
789                               color=COLOR_REFERENCIA,
                                    linewidths=2, zorder=3,
790                               label="Meta del sector")
791 
792         # El porcentaje y el estado escritos al lado de cada
                barra hacen que el
793         # estado nunca quede informado solamente por el color
794         for i in range(len(posiciones)):
795             grafico_metas.text(porcentajes[i] + 0.15,
                    posiciones[i], textos[i],
796                                va="center", fontsize=9,
```

```
                                color=COLOR_REFERENCIA)
797
798         grafico_metas.set_yticks(posiciones)
799         grafico_metas.set_yticklabels(sectores_por_merma)
800         grafico_metas.set_title("Merma real contra la meta de
               cada sector")
801         grafico_metas.set_xlabel("Merma sobre facturación (%)")
802         grafico_metas.set_xlim(0, max(porcentajes) * 1.5)
803         grafico_metas.grid(True, axis="x", color="#dddddd",
               linewidth=0.6)
804         grafico_metas.set_axisbelow(True)
805         grafico_metas.legend(loc="lower right")
806
807         # Este panel esta en pesos y el anterior en porcentaje:
               son medidas
808         # distintas, por eso van separadas. Un sector puede
               tener un porcentaje
809         # bajo y aun asi ser el que mas plata pierde, porque
               factura mucho
810         sectores_por_perdida = sorted(merma_por_sector,
811                                       key=lambda s:
                                          merma_por_sector[s])
812
813         perdidas = []
814
815         for sector in sectores_por_perdida:
816             perdidas.append(merma_por_sector[sector] / 1000000)
817
818         grafico_pesos.barh(sectores_por_perdida, perdidas,
819                            color=COLOR_FACTURACION, height=0.6,
                                  zorder=2)
820
821         for i in range(len(perdidas)):
822             grafico_pesos.text(perdidas[i], i, f"
                   {perdidas[i]:.1f}",
823                                va="center",
                                   color=COLOR_REFERENCIA)
824
825         grafico_pesos.set_title("Pérdida en dinero por sector")
826         grafico_pesos.set_xlabel("Merma (millones de $)")
827         grafico_pesos.set_xlim(0, max(perdidas) * 1.2)
828         grafico_pesos.grid(True, axis="x", color="#dddddd",
               linewidth=0.6)
829         grafico_pesos.set_axisbelow(True)
830
```

```
831             plt.tight_layout()
832             plt.show()
833 
834         #
                ------------------------------------------------------------
835         # 9. INFORME EN ARCHIVO DE TEXTO
836         #
                ------------------------------------------------------------
837         elif opcion == "9":
838             # lower() y replace() evitan mayusculas y espacios en el
                    nombre del
839             # archivo
840             nombre_archivo = "informe_" + sucursal.lower().replace("
                    ", "_") + ".txt"
841 
842             # "w" abre el archivo para escritura y lo crea si no
                    existe
843             informe = open(nombre_archivo, "w", encoding="utf-8")
844 
845             informe.write("=" * ANCHO + "\n")
846             informe.write("INFORME ANUAL DE FACTURACIÓN Y
                    MERMA".center(ANCHO) + "\n")
847             informe.write(f"Sucursal {sucursal}".center(ANCHO) +
                    "\n")
848             informe.write("=" * ANCHO + "\n\n")
849             informe.write(f"Generado por: {nombre} ({iniciales})\n")
850             informe.write(f"Sistema: control de gestión versión
                    {VERSION}\n")
851             informe.write(f"Autor del sistema: {AUTOR}\n\n")
852 
853             informe.write("RESUMEN ANUAL\n")
854             informe.write("-" * ANCHO + "\n")
855             informe.write(f"Facturación total:
                    ${total_anual:,.2f}\n")
856             informe.write(f"Merma total:
                    ${merma_anual:,.2f}\n")
857             informe.write(f"Merma sobre facturación:
                    {porcentaje_merma_anual:.2f} %\n")
858             informe.write(f"Promedio mensual:
                    ${promedio_mensual:,.2f}\n")
859             informe.write(f"Mejor mes:
                    {NOMBRES_MESES[mejor_mes]}\n")
860             informe.write(f"Peor mes:
                    {NOMBRES_MESES[peor_mes]}\n\n")
861 
```

```
862         informe.write("DETALLE POR SECTOR\n")
863         informe.write("-" * ANCHO + "\n")
864 
865         for sector in sectores_por_facturacion:
866             informe.write(f"{sector.ljust(12)} "
867                           f"facturó
                                 ${ventas_por_sector[sector]:>16,.2f}
                                   "
868                           f"merma
                                 {porcentajes_merma[sector]:>5.2f}%
                                 "
869                           f"{estados_sector[sector]}\n")
870 
871         informe.write("\nSECTORES QUE REQUIEREN REVISIÓN\n")
872         informe.write("-" * ANCHO + "\n")
873 
874         sectores_a_revisar = 0
875 
876         # Un sector sin ficha tampoco esta en OK, pero no se lo
              puede comparar
877         # contra una meta que no tiene, asi que se lo informa
              por separado para
878         # que de todas formas no pase inadvertido en el informe
879         for sector in sectores_por_facturacion:
880             if estados_sector[sector] == "SIN META":
881                 informe.write(f"{sector}:
                    {porcentajes_merma[sector]:.2f}% de "
882                               f"merma, sin ficha cargada en
                                    {ARCHIVO_SECTORES}. "
883                               f"No tiene meta asignada\n")
884 
885                 sectores_a_revisar = sectores_a_revisar + 1
886             elif estados_sector[sector] != "OK":
887                 meta = fichas_sectores[sector]["meta_merma"]
888                 desvio = porcentajes_merma[sector] - meta
889                 responsable =
                        fichas_sectores[sector]["responsable"]
890 
891                 informe.write(f"{sector}:
                    {porcentajes_merma[sector]:.2f}% contra "
892                               f"una meta de {meta:.2f}%
                                    ({desvio:+.2f} puntos). "
893                               f"Responsable: {responsable}\n")
894 
895                 sectores_a_revisar = sectores_a_revisar + 1
```

```
896
897         if sectores_a_revisar == 0:
898             informe.write("Ningún sector superó su meta de
                    merma\n")
899
900         informe.write("\n" + "=" * ANCHO + "\n")
901         informe.write("Fin del informe".center(ANCHO) + "\n")
902         informe.write("=" * ANCHO + "\n")
903
904         informe.close()
905
906         print(f"\n  Informe generado en el archivo
                {nombre_archivo}")
907         print(f"  Se listaron {sectores_a_revisar} sectores para
                revisar")
908
909     #
            ------------------------------------------------------------
910     # 10. EXPORTACION DE LOS DATOS PARA ANALISIS
911     #
            ------------------------------------------------------------
912     elif opcion == "10":
913         nombre_csv = "analisis_" + sucursal.lower().replace(" ",
                "_") + ".csv"
914
915         # Una observacion por fila y una variable por columna,
                que es el
916         # formato que esperan las herramientas de analisis: asi
                el archivo se
917         # puede abrir con Excel o leer con pandas y reagrupar
                por sector, por
918         # mes o por trimestre sin prepararlo antes
919         columnas = ["mes", "nombre_mes", "trimestre", "sector",
                "categoria",
920                     "responsable", "unidades", "precio_unitario",
921                     "facturacion", "unidades_merma", "merma",
922                     "porcentaje_merma", "meta_merma", "desvio",
                            "estado_anual"]
923
924         # El newline vacio es lo que pide el modulo csv al
                escribir: sin eso
925         # Windows deja una linea en blanco entre registro y
                registro
926         salida = open(nombre_csv, "w", newline="",
                encoding="utf-8")
```

```
927
928         # DictWriter es la contraparte de DictReader: recibe
                diccionarios y los
929         # escribe como filas, usando las columnas para ordenar
                cada dato
930         escritor = csv.DictWriter(salida, fieldnames=columnas)
931         escritor.writeheader()
932
933         for registro in detalle_filas:
934             escritor.writerow(registro)
935
936         salida.close()
937
938         print(f"\n  Datos exportados en el archivo {nombre_csv}")
939         print(f"  {len(detalle_filas)} filas y {len(columnas)}
                columnas")
940         print("  El archivo se puede abrir con Excel o leer con
                pandas para")
941         print("  seguir analizando los datos fuera de este
                sistema")
942
943     elif opcion != "0":
944         print(f"\n  La opción {opcion} no existe en el menú")
945
946
947 # ============================================================
948 # CIERRE DEL SISTEMA
949 # ============================================================
950
951 print()
952 print("=" * ANCHO)
953 print(f"Gracias por usar el sistema, {nombre}".center(ANCHO))
954 print(f"Control de gestión - Sucursal {sucursal}".center(ANCHO))
955 print("=" * ANCHO)
956 print()
```

Código 1: Examen_Final.py

↑ Volver al índice

6. Archivos de datos

6.1. sectores.csv

Archivo nuevo respecto del TP 4. Las metas se fijaron según el riesgo de cada rubro: los sectores de mercadería no perecedera admiten menos de medio punto de merma, mientras que la verdulería, que trabaja con productos que se descartan en pocos días, admite hasta cinco puntos.

```
sector,responsable,meta_merma,categoria
Almacen,Marta Gómez,0.40,No perecedero
Bebidas,Diego Ruiz,0.25,No perecedero
Carniceria,Hugo Paz,2.80,Perecedero
Fiambreria,Ana Sosa,2.50,Perecedero
Verduleria,Luis Vera,5.00,Perecedero
Limpieza,Sofía Díaz,0.20,No perecedero
```

Código 2: sectores.csv

6.2. ventas.csv

Es el mismo archivo del Ejercicio 14 del TP 4, con 72 registros. Se transcriben las primeras filas a modo de muestra.

```
mes,sector,unidades,precio_unitario,unidades_merma
1,Almacen,48600,1200,197
1,Bebidas,19440,1800,52
1,Carniceria,9720,6500,394
1,Fiambreria,6480,4800,219
1,Verduleria,16200,1400,1422
1,Limpieza,12960,2200,17
2,Almacen,42300,1250,165
...
```

↑ Volver al índice